

Course Name: EG 212, Computer Architecture - Processor Design

MIPS Architecture Design

Pratham Vyas, Soham Patil, Yashamit Dawane

BC2025078

BC2025094

BC2025119

Introduction

MIPS (Microprocessor without Interlocked Pipelined Stages) is a family of reduced instruction set computer (RISC) instruction set architectures (ISA) developed by MIPS Computer Systems, now MIPS Technologies, based in the United States. Computer architecture courses in universities and technical schools often study MIPS architecture. The architecture greatly influenced later RISC architectures such as Alpha. In March 2021, MIPS announced that the development of the MIPS architecture had ended as the company is making the transition to RISC-V.

Processor

For this assignment, a single C++ program (Factorial Calculation) was converted into MIPS assembly and verified using the MARS Assembler.

Additionally, we coded a Non-Pipelined MIPS Processor in Python to simulate hardware-level execution. The processor takes raw 32-bit binary strings as instruction memory (`machine_code.txt`) and data memory (`data.txt`), mapping perfectly to MARS's binary text dump formats. The code is modularly separated into the five classic stages of MIPS execution, allowing for easier debugging and readability.

Algorithms Implemented:

Factorial: Takes an integer n as input from memory and calculates its factorial ($n!$), storing the final result back into the data memory segment.

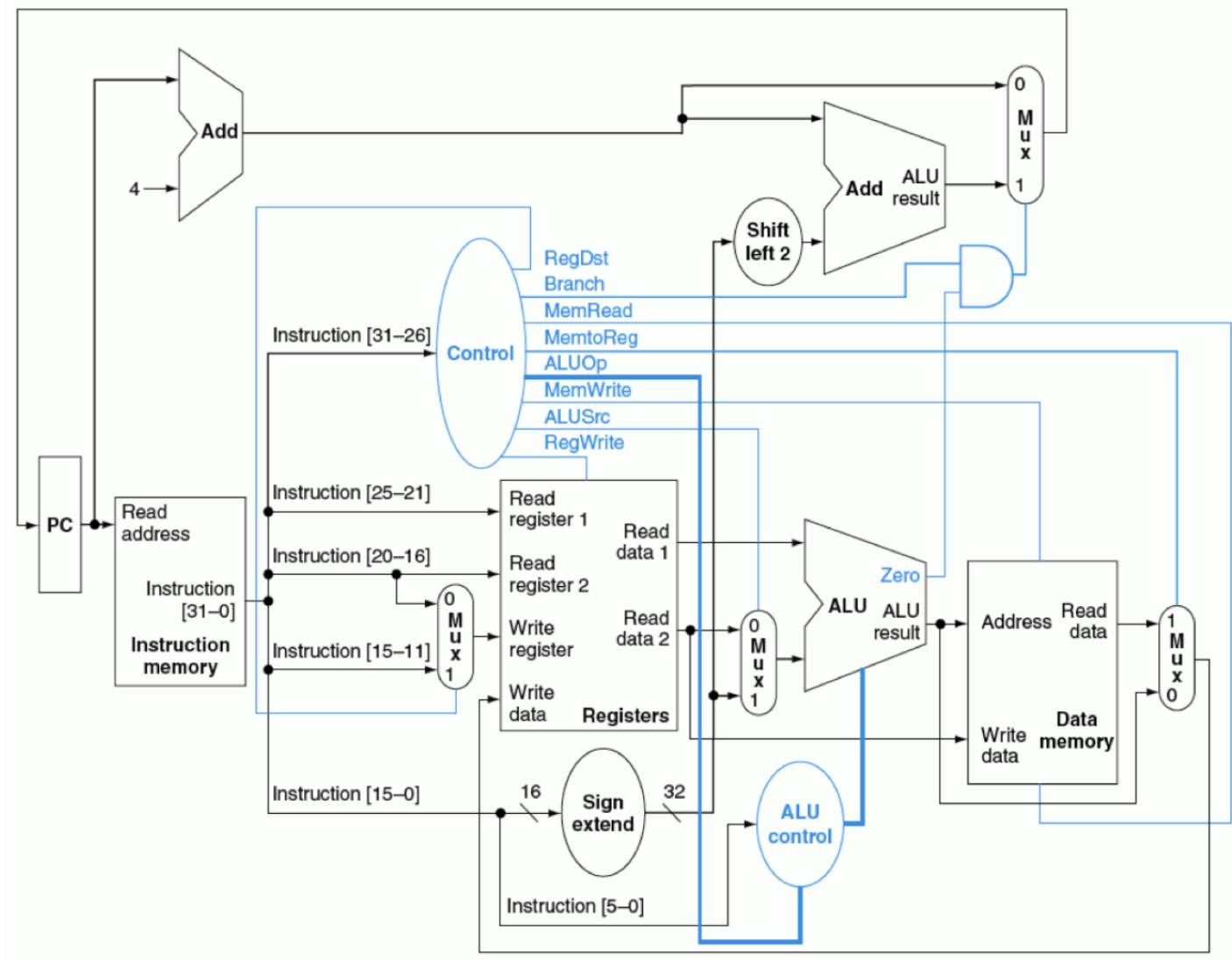


Figure 1: MIPS Architecture

Factorial Algorithm:

This program takes a sequence of instructions to calculate the factorial of a predefined number (e.g., 5) and stores it sequentially. **Find original cpp code below:**

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int input_num = 5;
6      int result = 1;
7
8      while (input_num != 0) {
9          result = result * input_num;
10         input_num--;
11     }
12
13     // Store result in memory (represented here as variable assignment)
14     int final_memory_store = result;
15     return 0;
16 }

```

MIPS Assembly Implementation

```

1  .data
2  input_num: .word 5      # The value 5 is stored at 0x10010000
3  result:    .word 0      # Space for result at 0x10010004
4
5  .text
6  .globl main
7
8  main:
9      # 1. LOAD ADDRESS MANUALLY
10     # Instead of 'la', we use 'lui' and 'ori' to form 0x10010000
11     # This aligns with MIPS encoding formats [cite: 1926]
12     lui    $t1, 0x1001    # Load upper 16 bits: $t1 becomes 0x10010000
13     ori    $t1, $t1, 0    # Lower 16 bits are 0. $t1 remains 0x10010000 [cite: 1754]
14
15     # 2. LOAD VALUE
16     # Load word from the address stored in $t1
17     lw     $t0, 0($t1)    # $t0 = 5 [cite: 1736]
18
19     # 3. INITIALIZE RESULT
20     # Use ADDI to set $t2 to 1 (0 + 1)
21     addi   $t2, $zero, 1  # $t2 = 1 [cite: 1914]
22
23 loop:
24     # 4. BRANCH CONDITION
25     # If $t0 is 0, we are finished
26     beq    $t0, $zero, end_loop # [cite: 1767]
27
28     # 5. MULTIPLY
29     # Multiply result by current number
30     mul    $t2, $t2, $t0
31
32     # 6. DECREMENT
33     # Use ADDI with a negative immediate to subtract
34     addi   $t0, $t0, -1   # $t0 = $t0 - 1 [cite: 1917]
35
36     # 7. JUMP
37     # Jump back to the start of the loop
38     j      loop          # [cite: 1964]
39
40 end_loop:
41     # 8. STORE RESULT
42     # Store the value in $t2 to the address in $t1 with offset 4
43     # Address = 0x10010000 + 4 = 0x10010004
44     sw     $t2, 4($t1)    # [cite: 1736]
45
46     # 9. EXIT
47     # Basic syscall to stop the program
48     addi   $v0, $zero, 10
49     syscall

```

Defining Each Component Used:

Rather than physical classes, our simulator defines the processor components through strict data arrays and functional stages mimicking hardware logic paths.

- **Register File (`registers[32]`):** Creates an array of 32 elements representing general-purpose registers, capable of reading and writing 32-bit signed values. Register \$0 is strictly enforced to 0.
- **Data Memory (`data_memory`):** An array representing RAM, populated initially by parsing the MARS `data.txt` dump.
- **Instruction Fetch (`fetch`):** Reads the current instruction from `instruction_memory` using the Program Counter (PC) and increments the PC by 4.
- **Instruction Decode (`decode`):** Simulates the Control Unit and Splitter by slicing the 32-bit binary string into functional segments: Opcode (bits 31-26), RS, RT, RD, Shift Amount, and Funct (bits 5-0).
- **Execute (`execute`):** Acts as the ALU and ALU Control. It maps the decoded operations to arithmetic execution and calculates branching/jump target addresses based on the offset.
- **Memory Access (`memory_access`):** Handles load/store logic. Identifies LW and SW opcodes to interact with the `data_memory` array and simultaneously syncs physical updates to `data.txt`.
- **Write Back (`write_back`):** Commits the ALU result or Memory Read Data back into the destination register (\$rd or \$rt).

Processor:

Main Execution Loop: This runs the processor loop until termination conditions are met.

```
processor.py | Execute
import sys

# =====
# MIPS Processor Simulation (Fixed for J)
# =====

# --- Hardware Components ---
registers = [0] * 32      # 32 general-purpose registers ($0 to $31)
data_memory = [0] * 1024 # 1KB Data Memory (Word-indexed internally)
instruction_memory = []   # Instruction Memory
pc = 0                   # Program Counter

# --- Configuration ---
DATA_START_ADDR = 0x10010000
TEXT_BASE = 0x00400000    # <-- typical assembler text base (we map J targets against this)
max_data_index = 0        # Tracks how many lines to write back to data.txt

# --- Helper: Sign Extension ---
> def sign_extend(value, bits):...

# --- File Loaders & Writers ---
> def load_instructions(filename):...
> def load_data(filename):...
> def sync_data_to_file(filename):...

# --- Stage 1: FETCH ---
> def fetch():...

# --- Stage 2: DECODE ---
> def decode(instr):...

# --- Stage 3: EXECUTE ---
> def execute(decoded, next_pc):...

# --- Stage 4: MEMORY ACCESS ---
> def memory_access(decoded, alu_result):...

# --- Stage 5: WRITE BACK ---
> def write_back(decoded, alu_result, mem_read_data):...

# =====
# Main Execution Block
# =====
> if __name__ == "__main__":...
```

Opcode and Funct Mapping: Instead of external Enumerations, the processor decodes logic dynamically using direct binary string comparison, mirroring hardware logic gates checking specific wire states:

```
1  # R-Type
2  if op == "000000":
3      if funct == "100000": # ADD
4          alu_result = rs_val + rt_val
5      elif funct == "001100": # SYSCALL
6          if registers[2] == 10:
7              return None, False, 0
8      # I-Type
9      elif op == "100011": # LW
10         alu_result = rs_val + sign_extend(imm, 16)
```

